

ATOM

AI TOTAL OPTIMIZATION MANAGER

ATOM 통합 산출물

AI 상품 추천 · 동적 프라이싱 · 쿠폰 최적화 시스템

개발팀 & 현업팀 공용 산출물 북 (Delivery Book)

작성 위레이저 (Weraser)

작성일 2026-06-19

문서 상태 초안 (Draft)

인도 범위 추천·가격·쿠폰 일배치 · 추천·쿠폰 API · AI 모델 라이브러리 · 매장 메뉴 크롤러 · 경쟁사
가격 크롤러 (전체 전달)

§ 문서 정보

본 산출물은 ATOM의 **현업 안내**와 **개발 기술 명세**를 하나로 묶은 문서입니다. 한 권 안에서 비개발 현업 담당자(1부)와 개발팀(2부)이 각자 필요한 부분을 볼 수 있도록 구성했습니다.

항목	내용
시스템명	ATOM (AI Total Optimization Manager): 차별화상회 플랫폼용 AI 추천·프라이싱·쿠폰 시스템
구성	1부 현업용 안내서 + 2부 개발용 기술 산출물 + 부록(인도 범위·연동 인터페이스·보안)
상태 / 일자	초안 / 2026-06-19
대상 독자	1부: 차별화상회 현업 담당자 / 2부: 위레이저·스마트 푸드 네트워크 개발팀

보안 분류: 대외비

본 문서에는 시스템 구조·운영 정보가 포함됩니다. DB 비밀번호 · API 키 · Basic Auth · Slack Webhook 등 모든 자격증명 값은 **마스킹**되어 있으며, 실제 자격증명은 **AWS Secrets Manager(atom/{env}/config)**에서 로드되고 별도 보안 채널로 전달됩니다(부록 C 참조). 본 문서는 권한 있는 담당자에게만 공유하십시오.

§ 목차

제 1 부 · 현업용 안내서 **현업** : 기술 용어 없이 ‘무엇을·어떻게·언제’

- 1 ATOM이 하는 일
- 2 하루 흐름: 언제 무슨 일이 일어나나
- 3 추천은 어떻게 정해지나
- 4 가격은 어떻게 정해지나
- 5 쿠폰 5종: 누구에게, 왜
- 6 고객·상품은 어떻게 나누나
- 7 문자(LMS) 안내
- 8 현업이 할 일: 오후 가격 검수 승인 ★
- 9 자주 묻는 질문 & 현업 용어풀이

제 2 부 · 개발용 기술 산출물 **개발** : 아키텍처·스키마·수식·운영

- 10 솔루션 개요 & 책임 경계
- 11 시스템 아키텍처 (컨텍스트·컨테이너·컴포넌트)
- 12 레포 구성 & 인프라

-
- 13** 데이터 인터페이스 (S3·SQS·API)
 -
 - 14** DB 스키마: 원천 데이터(차별화상회)
 -
 - 15** DB 스키마: 산출 데이터(Weraser)
 -
 - 16** ERD: 핵심 테이블 관계
 -
 - 17** 모델 상세 (A · B · C)
 -
 - 18** 배치 전체 로직 (3-Phase + 스케줄러)
 -
 - 19** 가격 · 쿠폰 정책
 -
 - 20** 운영: 실행·배포·모니터링·장애 Runbook
 -

부록

-
- A** 인도 범위: 무엇이 전달되고 무엇이 아닌가
 -
 - B** 연동 인터페이스 (프로파일링·크롤링·승인)
 -
 - C** 자격증명·시크릿 관리 (마스킹 안내)
 -
 - D** 통합 용어집
 -

PART I

현업용 안내서

기술 용어 없이, ATOM이 **무엇을 / 어떻게 / 언제** 하는지 설명합니다. 기획·운영·마케팅·영업 담당자를 위한 부분입니다. 데이터 구조·계산식·서버 운영 등 기술 상세는 **제 2 부 (개발용)**를 참고하세요.

대상 · 차별화상회 현업 담당자

1 ATOM이 하는 일

ATOM은 차별화상회 고객(식당·카페 등)의 데이터를 **매일 자동으로 분석**해서, 고객마다 **딱 맞는 추천 상품·가격·쿠폰**을 만들어 차별화상회 플랫폼에 공급하는 시스템입니다.

기능	쉽게 말하면	고객이 느끼는 것
① 상품 추천	이 가게가 살 만한 상품을 골라줍니다	“우리 가게에 딱 맞는 상품이 보이네”
② 가격 조정	가격을 조금 올린 뒤, 쿠폰으로 다시 깎아줍니다	“쿠폰 받으니 결국 싸게 사네”
③ 쿠폰 발급	고객마다 다른 할인 쿠폰을 줍니다	“나한테 맞는 할인이 들어왔네”

핵심 아이디어

정가를 조금 올리고(가격 인상), 그만큼을 쿠폰으로 돌려줘서 고객이 실제로 내는 “**최종 결제가(혜택가)**”는 **합리적으로** 유지합니다. 그러면서 고객마다 다른 추천과 할인을 제공합니다.

2 하루 흐름: 언제 무슨 일이 일어나나

ATOM은 **매일 정해진 시간에 자동으로** 돌아갑니다.

시간	하는 일	현업 관여
아침 7시쯤	차별화상회가 고객·상품·주문 데이터를 보내줌	-

아침 8시	데이터 분석 시작 (고객 분류, 추천 점수, 가격 계획 만들기)	-
오전 10시	고객에게 추천·쿠폰 안내 문자 발송 (1차)	-
오후 2시	그날 바뀔 가격을 정리해 검수 메일 발송	담당자 승인 필요
오후 3시	추천·쿠폰 안내 문자 발송 (2차)	-
오후 4시 10분	(승인되면) 가격 변경 + 기본 쿠폰 반영	-
오후 6시 10분	고객별 맞춤 쿠폰·장바구니 쿠폰 발급	-
다음 날	차별화상회 앱/웹에 추천·가격·쿠폰 노출	-

주말 차이

토·일에는 가격을 새로 만들지 않고 **금요일 가격을 그대로 유지**합니다. (추천·쿠폰은 주말에도 동작) 그래서 주말에는 가격 검수 승인도 없습니다.

3 추천은 어떻게 정해지나

고객마다 “이 상품을 추천할 가치”를 **점수로 매겨서**, 점수 높은 상품을 추천합니다. 점수는 아래 정보들을 종합합니다.

- **이 고객이 예전에 무엇을 샀는지**: 자주 사는 상품, 다시 살 때가 된 상품
- **함께 잘 팔리는 상품**: A를 사는 고객은 보통 B도 삼
- **이 가게가 파는 메뉴**: 메뉴를 보고 필요한 식자재를 추측
- **같은 업종에서 인기 있는 상품**: 아직 구매 이력이 없는 새 고객용

추천은 두 종류로 나뉩니다.

종류	의미
재구매 추천	예전에 사본 상품을 “다시 살 때가 됐어요” 하고 권함
신규 추천	아직 안 사본 상품을 “이것도 어울려요” 하고 권함

4 가격은 어떻게 정해지나

ATOM은 “**가격을 올리고 쿠폰으로 돌려주는**” 방식을 씁니다.

1. 상품 판매가를 **10~15% 올립니다.** (가격 인상)
2. 올린 만큼을 **기본 쿠폰으로 깎아줍니다.**
3. 그래서 고객이 실제로 내는 **최종 결제가는 합리적으로** 유지됩니다.

이렇게 하면 “정가는 높아 보이지만, 쿠폰을 받아 싸게 산다”는 혜택 느낌을 줄 수 있습니다. 경쟁사(다른 식자재물)보다 우리가 비싸면 가격을 조금 낮추고, 우리가 더 싸면 굳이 더 내리지 않는 식으로 **경쟁사 가격도 참고**해 조정합니다.

검수 승인 연결

매일 오후 2시에 “그날 바뀔 가격 목록”을 담당자에게 메일로 보내고, 담당자가 승인해야 오후 4시 10분에 실제로 반영됩니다. (8번 항목 참고)

5 쿠폰 5종: 누구에게, 왜

쿠폰	누구에게	왜
기본 쿠폰	모든 상품	가격 인상분을 상쇄해 합리적 가격을 보여주려고
재구매 쿠폰	예전에 산 상품	다시 살 때 할인으로 재구매 유도
신규 상품 쿠폰	아직 안 사본 상품	새 상품을 처음 사보도록 할인
장바구니 쿠폰	장바구니 전체	일정 금액 이상 사면 추가 할인 (객단가 상승 유도)
이탈 고객 쿠폰	한동안 안 산 고객	다시 돌아오도록 복귀 유도

알아두실 점

- 장바구니·이탈·VIP 쿠폰은 “직배송 고객”에게만 나갑니다. (택배 고객 제외)
- 맞춤 상품 쿠폰(재구매·신규)은 직배송·택배 고객 모두에게 나갑니다.
- 장바구니 쿠폰은 고객마다 정해진 요일에 주 1회 발급되고, 받은 날부터 7일간 쓸 수 있습니다.
- VIP 쿠폰: 우량 고객(많이·꾸준히 사는 고객)에게 큰 금액의 장바구니 쿠폰을 추가로 줍니다.

6 고객·상품은 어떻게 나누나

고객 분류

- 프랜차이즈 고객 / 일반 식당으로 먼저 나눕니다. (프랜차이즈는 본사 기준 추천이 가능)
- 고객이 활발한지 / 점점 안 오는지(이탈), 할인에 민감한지 등을 분석해 맞춤 전략을 정합니다.

상품 중요도 4단계 (한 고객 안에서)

한 고객이 산 상품들을 그 고객이 돈을 많이 쓴 순서로 4등분합니다.

단계	의미	쿠폰
1순위 (TOP)	그 고객이 가장 많이 사는 핵심 상품	할인 크게
2순위 (KEY)	자주 사는 상품	할인 크게
3순위 (STANDARD)	보통 상품	할인 보통
4순위 (TAIL)	가끔 사는 상품	할인 작게

예시

어떤 식당이 6개월간 산 상품을 매출 많은 순으로 줄 세워, 위에서 25%는 1순위, 다음 25%는 2순위... 식으로 나눕니다. **매출 1등 상품은 항상 1순위**가 됩니다.

7 문자(LMS) 안내

ATOM은 고객에게 **추천 상품과 쿠폰을 문자로 안내**합니다.

- 추천 상품 상위 5개 + 안내 메시지 + 쿠폰 정보를 담아 보냅니다.
- 고객마다 **정해진 요일·시간(오전 10시 또는 오후 3시)에 주 1회** 발송합니다. (고객의 평소 주문 패턴을 보고 반응이 좋을 시 간으로 정함)
- 추천할 상품이 없는 고객에게는 보내지 않습니다.
- 문자도 **직배송 고객**을 대상으로 발송합니다.

8 현업이 할 일: 오후 가격 검수 승인 ★

★ ATOM에서 현업 담당자가 직접 해야 하는 유일한 작업입니다.

1. **오후 2시쯤 검수 메일**을 받습니다.: 그날 바뀔 “상위 500개 상품”의 가격 변경 예정 목록입니다.
2. 목록을 확인합니다.: 이상하게 높거나 낮은 가격이 있는지 봅니다.
3. **문제없으면 “승인”** → 오후 4시 10분에 가격·기본 쿠폰이 플랫폼에 자동 반영됩니다.
4. **이상하면 “반려”** → 그날 가격 변경은 적용되지 않습니다.
5. **마감은 오후 5시 30분**. 그때까지 승인하지 않으면 그날 가격 변경은 자동으로 보류됩니다.

주말

주말에는 가격을 바꾸지 않으므로 검수 승인이 없습니다.

9 자주 묻는 질문 & 현업 용어풀이

자주 묻는 질문

Q. 주말에는 왜 가격이 그대로인가요?

주말에는 가격을 새로 만들지 않고 금요일 가격을 유지하는 정책입니다. 그래서 주말 동안 가격·기본 쿠폰은 금요일 것과 같습니다.

Q. 왜 어떤 고객에게는 장바구니 쿠폰이 안 가나요?

장바구니 쿠폰은 **직배송 고객에게만** 발급됩니다. 택배로 받는 고객은 대상이 아닙니다.

Q. 같은 고객인데 어떤 상품은 할인이 크고 어떤 건 작아요.

그 고객이 **많이 사는 핵심 상품일수록 할인을 크게**, 가끔 사는 상품은 작게 주도록 설계돼 있습니다. (6번 상품 중요도 4단계 참고)

Q. 추천이 매일 바뀌나요?

네. 매일 아침 그날의 데이터로 다시 계산하므로, 고객의 최근 구매·접속에 따라 추천과 가격이 매일 갱신됩니다.

Q. 검수 승인을 깜빡하면요?

오후 5시 30분까지 승인하지 않으면 그날 가격 변경은 보류되고, 기존 가격이 유지됩니다. 시스템이 멈추지는 않습니다.

현업 용어풀이

자주 보이는 말	쉬운 뜻
가격 인상 (Up-price)	판매가를 10~15% 올리는 것 (쿠폰으로 다시 깎아줌)
혜택가 / 최종 결제가	쿠폰까지 적용한 뒤 고객이 실제 내는 가격
재구매 추천 / 신규 추천	예전에 산 상품을 다시 권함 / 안 사본 상품을 새로 권함
상품 중요도 1~4순위	그 고객이 돈을 많이 쓴 순서로 나눈 4단계 (TOP/KEY/STANDARD/TAIL)
직배송 / 택배	고객이 받는 배송 방식 (장바구니·이탈 쿠폰은 직배송만)
프랜차이즈(FC)	같은 브랜드로 여러 지점을 운영하는 음식점
검수 승인	오후 2시 가격 변경 목록을 담당자가 확인·승인하는 일
이탈 고객 / VIP 고객	한동안 주문이 없는 고객 / 많이·꾸준히 사는 우량 고객
손익분기(BEP)	쿠폰을 줘도 손해 안 보는 최소 주문 금액
이익률(GP)	판 돈에서 원가·할인을 뺀 뒤 남는 이익의 비율

PART II

개발용 기술 산출물

시스템 아키텍처, DB 스키마, 모델 수식, 배치 로직, 운영 절차를 다룹니다. 위레이저·스마트 푸드 네트워크 양측 개발팀을 위한 부분입니다.

대상 · 위레이저 / 스마트 푸드 네트워크 개발팀

10 솔루션 개요 & 책임 경계

ATOM은 차별화상회 플랫폼의 고객 데이터를 매일 분석해 **고객별 맞춤 추천 상품·가격·쿠폰**을 자동 산출, 차별화상회에 공급하는 AI 시스템이다.

3대 기능

기능	설명	핵심 산출
상품 추천	구매이력·메뉴·업종 분석 → 살 만한 상품 점수화	A-6 final_score, recommend_type
동적 프라이싱	정가/판매가 10~15% 랜덤 인상(Up-price)	planned_prices (판매가·기본쿠폰)
쿠폰 최적화	기본·개인화·장바구니·VIP·이탈 쿠폰 고객별 발급	coupon_records, cart_coupon_results

핵심 메커니즘

가격을 올린 만큼(Up-price)을 쿠폰으로 돌려주어 **혜택가 = 인상 판매가 - 쿠폰 할인**을 만들고, 추천·할인을 고객별로 개인화한다.

책임 경계 (차별화상회 ↔ 위레이저)

주체	책임
차별화상회	데이터 원천 제공(S3·SQS), 플랫폼 반영(가격·쿠폰·추천 노출), 가격 검수 승인, 발급 API 제공
Weraser	추천·프라이싱·쿠폰 산출 엔진 개발·운영, 프로파일링·크롤링 데이터 가공, 일배치 운영

데이터 분리 원칙

차별화상회 **원천 데이터**(고객·주문·상품...)와 Weraser **산출 데이터**(프로필·추천·쿠폰...)는 테이블 단위로 분리한다(§ 14·§ 15). ATOM은 원천 데이터를 수정하지 않는다.

11 시스템 아키텍처

ATOM은 세 주체의 AWS 환경에 걸쳐 동작한다. 추천·가격·쿠폰을 산출하는 핵심(API 서버·일배치·RDS)은 차별화상회 AWS 계정에서 운영되고, 메뉴 식자재 추출(프로파일링)과 관리 대시보드는 위레이저 AWS, 경쟁사 가격·매장 메뉴 수집(크롤러)은 별도 관리 VPC에서 동작한다.

11.1 전체 구성과 데이터 흐름

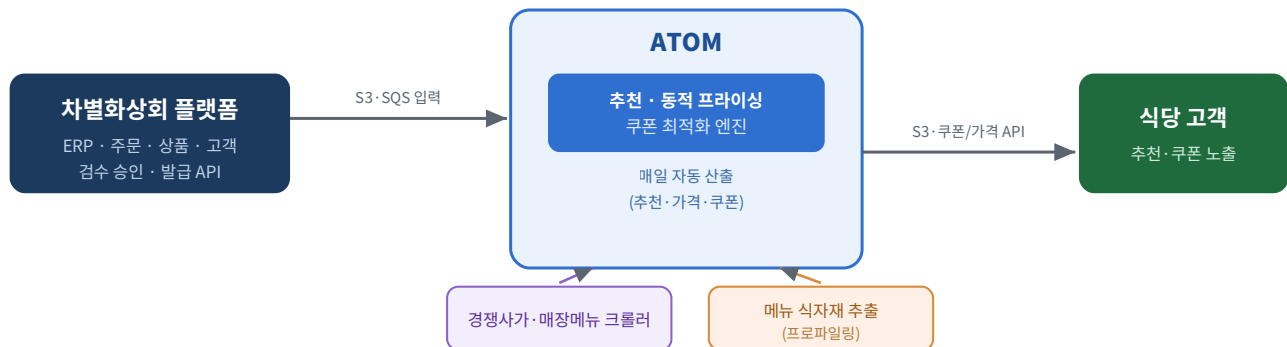


그림 11-1. 전체 구성. 데이터 원천 제공과 플랫폼 반영은 차별화상회, 추천·가격·쿠폰 산출은 ATOM이 담당한다.

11.2 네트워크 · VPC 피어링 구성 (운영 기준)

ATOM 핵심은 차별화상회 AWS 계정(0999900259356)에서 동작한다. 위레이저 프로파일링·대시보드와는 **VPC 피어링(사설 IP)**으로 연결되고, 위레이저 대시보드 백엔드 호출만 공인 인터넷(NAT)을 경유한다.

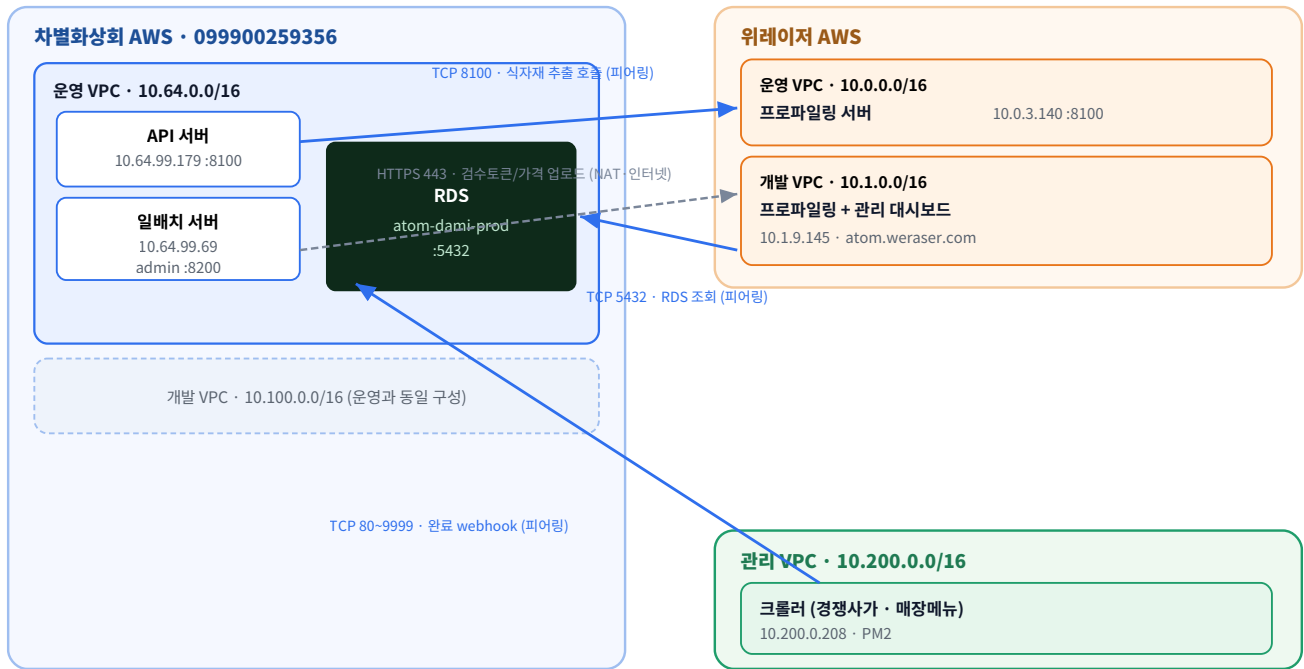


그림 11-2. 네트워크·VPC 피어링(운영 기준). 파란 실선은 사실 IP 피어링(보안그룹 허용), 회색 점선은 공인 인터넷(NAT) 경우. 개발 환경도 동일 구조로 구성된다.

구간	방향	포트	경로	용도
API 서버 → 프로파일링	차별화상회 → 위레이저	8100	피어링	메뉴 식자재 추출(LLM) 호출
프로파일링/대시보드 → RDS	위레이저 → 차별화상회	5432	피어링	대시보드의 ATOM RDS 직접 조회
크롤러 → API(ALB)	관리 VPC → 차별화상회	80~9999	피어링	크롤링 완료 /api/webhook/crawl-complete
일배치 → 대시보드 백엔드	차별화상회 → 위레이저	443	인터넷(NAT)	검수 토큰 생성 · 가격 데이터 업로드

접근 통제

차별화상회 내부 서버는 보안그룹 참조(SG referencing)로 RDS에 접근하고, 위레이저 서버는 CIDR(사실 IP) 허용으로 통제한다. 운영 RDS는 위레이저 운영 VPC 대역(10.0.0.0/16)을 사전 허용해 두었다.

11.3 ATOM 제공 서비스 구성

ATOM이 제공·운영하는 서비스 묶음이다. 일배치가 중심에서 모든 산출을 오케스트레이션하고, 모델·프로파일링·크롤링 결과를 입력으로 사용한다.



그림 11-3. ATOM 서비스 구성. 왼쪽(모델·식자재 추출·크롤링)은 일배치 입력, 오른쪽(API·산출물·발급)은 차별화상회로 나가는 출력.

11.4 컨테이너 · 컴포넌트

컴포넌트	역할
추천·쿠폰 API 서버	추천/쿠폰 조회 API, 신규·수정 고객 실시간 처리(FIFO), 프로파일링 결과 수신(/api/v1/profiling/save), 크롤링 완료 webhook 수신. 관리 API(:8200)로 검수 승인 게이트 제공
일배치 서버	스케줄러(30초 폴링)가 3-Phase를 시각에 맞춰 실행, S3 이벤트 소비기가 원천 데이터를 적재. 산출의 중심
AI 모델 라이브러리	연관구매·구매주기·서브클러스터·장바구니쿠폰 모델. 일배치에 내장(editable) 설치되어 함께 호출
메뉴 식자재 추출(프로파일링)	크롤링한 매장 메뉴를 LLM으로 식자재로 변환. 결과는 API를 통해서만 RDS에 저장
크롤러	경쟁사 가격(식품·배민상회), 매장 메뉴(네이버·카카오·다이닝코드·식신) 수집 (§ 12.6 · § 12.7)
데이터 저장소	PostgreSQL RDS(atom_prod 산출 / crawler_prod 크롤링), S3(입력·산출), SQS(이벤트), Secrets Manager(자격증명)

12 구성요소 · 인프라

12.1 구성요소 & 인도 범위

구성요소 (역할)	주요 기능	인도
추천·가격·쿠폰 일배치	3-Phase 일배치(고객 분석 · 추천 · 가격 · 쿠폰 산출/발급) + S3 이벤트 수신. 운영 메인 시스템	전체 전달
추천·쿠폰 API 서버	추천/쿠폰 조회 API · 프로파일링 결과 수신 · 신규/수정 고객 실시간 처리	전체 전달
AI 모델 라이브러리	연관구매 · 구매주기 · 장바구니쿠폰 · 서브클러스터 등 ML 모델 모음 (일배치에 내장 설치)	전체 전달
매장 메뉴 크롤러	지도·맛집 플랫폼에서 고객 매장 메뉴·평점 수집 (상세 § 12.6)	전체 전달
경쟁사 가격 크롤러	경쟁 채널(식품·배민상회) 동일상품 가격 수집·환산 (상세 § 12.7)	전체 전달
메뉴 프로파일링 서비스	매장 메뉴 → 식자재 추출 (C 모델, Gemini)	연동 인터페이스 (Weraser 운영)

관리 대시보드 (승인 게이트)	가격 · LMS 검수·승인	연동 인터페이스 (Weraser 운영)
------------------	----------------	------------------------------

인도 · 배포 방식

전체 전달 구성요소는 각각 GitHub 저장소로 전달한다. 일배치·API·크롤러는 AWS **CodePipeline**(*-dev/*-prod) → CodeDeploy로 배포되며, 로컬에서는 `git push`로 반영한다. AI 모델 라이브러리는 일배치에 내장(editable) 설치되어 함께 전달된다. 메뉴 프로파일링 서비스·관리 대시보드는 Weraser가 운영하며, 연동 규격(부록 B)만으로 동작한다.

12.2 기술 스택

영역	스택
백엔드 공통	Python 3.12 · FastAPI · Uvicorn · SQLAlchemy 2.x(async + asyncpg) · PostgreSQL 16 · boto3 · httpx · pandas/numpy · scikit-learn
패키지/포맷	uv (pip 대체) · ruff (line-length 120)
프로파일링(C)	google-genai (Gemini 2.5 Flash) · rapidfuzz + jamo (한글 퍼지매칭)
인프라	AWS EC2 · RDS(Aurora PostgreSQL) · S3 · SQS · Secrets Manager · CodePipeline/CodeDeploy

12.3 런타임 / EC2

환경	배치 EC2	API EC2	접근
PROD	i-0ebecdd5984cc878a (10.64.99.69)	i-004ce979a24578696 (10.64.99.179)	스마트 푸드 네트워크 VPC(Private) · SSM <code>--profile sfn</code>
DEV	i-0cd5ebf17b3feb8f2 (10.100.98.199)	i-0f8e537d027b8e58f (10.100.99.173)	SSM <code>--profile sfn</code>

서비스 (systemd)	내용
배치: <code>atom-scheduler</code>	스케줄러 + admin API(:8200)
배치: <code>atom-s3-consumer</code>	S3 이벤트(SQS) 소비
API: <code>atom-api</code> (:8100)	추천/쿠폰/프로파일링 API
API: <code>atom-sqs</code>	신규/수정 고객 SQS(FIFO) 소비

※ 프로파일링/대시보드 EC2는 Weraser VPC(Public, SSH). 크롤러 EC2는 PM2(menu :3000 / product :3001) + 다이닝코드 AWS Lambda.

12.4 DB (PostgreSQL RDS · Aurora)

환경	호스트	DB
PROD	<code>atom-dami-prod.cluster-c9eqiy6sq0dc.ap-northeast-2.rds.amazonaws.com:5432</code>	<code>atom_prod</code> / <code>crawler_prod</code>
DEV	<code>atom-dami-dev.c9eqiy6sq0dc.ap-northeast-2.rds.amazonaws.com:5432</code>	<code>atom-dev</code> / <code>crawler_dev</code>

접속 자격증명

DB 접속정보는 **AWS Secrets Manager**(`atom/{env}/config`, `atom/{env}/db`)에서 로드한다. 비밀번호 등 실값은 본 문서에 포함하지 않으며 별도 보안 채널로 전달한다(부록 C). 스마트 푸드 네트워크 VPC 내부에서만 접근 가능, 백업은 자동 스냅샷 7일 (PITR).

12.5 크롤링 시스템 개요

두 크롤러(**매장 메뉴 · 경쟁사 가격**)는 단일 저장소의 두 앱으로 구성된다. 모두 **TypeScript/Node.js** 기반이며 **BullMQ + Redis** 작업 큐와 **Prisma(PostgreSQL)**를 사용한다. EC2 위에서 PM2로 상시 가동되고, 메뉴 크롤러의 다이닝코드 수집만 AWS Lambda로 분리된다.

공통 항목	내용
읽는 DB	<code>atom_{env}</code> (고객·상품 마스터): READ ONLY (<code>pg.Pool</code> 직접 조회)
쓰는 DB	<code>crawler_{env}</code> : Prisma (크롤링 결과 저장)
완료 통지	수집 완료 시 ATOM API <code>POST /api/webhook/crawl-complete (type=menu price)</code> 콜백
실행 호스트	EC2(<code>atom-crawler-{env}</code>) · PM2 (메뉴 :3000 / 가격 :3001), 다이닝코드는 AWS Lambda
자격증명	AWS Secrets Manager(<code>atom/{env}/db</code> , <code>atom/{env}/config</code>)에서 로드 (마스킹·부록 C)

12.6 매장 메뉴 크롤러

고객(가게) 매장명·주소를 입력받아 지도·맛집 플랫폼에서 메뉴·평점·리뷰를 수집한다. **네이버 → 카카오 → 다이닝코드 → 식신** 4단계 폴백(cascade)으로 동작하며, 메뉴가 **5개 초과** 수집되면 즉시 종료하고 부족·실패 시 다음 소스로 넘어간다.



그림 12-1. 메뉴 크롤러 4단 캐스케이드. 각 소스는 독립 큐+워커로 동작하며, 메뉴 충분 시 즉시 종료·Webhook 발사.

항목	내용
API / 큐	Express(:3000) · 소스별 독립 BullMQ 큐(naver/kakao/diningcode/sikshin)
수집 기술	네이버·카카오·식신 = Playwright+Stealth / 다이닝코드 = AWS Lambda(Puppeteer+Chromium, 멀티리전 라운드 로빈, <code>x-api-key</code>) / 주소변환 = Kakao Local API
가동 방식	① 외부 트리거: <code>POST /api/crawl/customers</code> (외부 파이프라인이 고객 ID 배열 전달) ② 워커 24/7 listen 루프(60초 폴링, 큐가 비면 가장 오래 전 수집된 고객부터 자동 재적재). 고정 cron 없음: self-feeding
소스별 동시성	naver 3(분당 10) · kakao 4(40) · diningcode 6(60) · sikshin 2(20)
매칭 판정	이름 유사도 ≥ 0.7 + 주소 일치(도로명/지번), 비음식 카테고리 제외, 폐업(NTS <code>closed</code>) 제외
입력	<code>customers</code> (atom, 매장명·주소) · <code>nts_validations</code> (crawler, 사업자 상태)
출력(crawler DB)	<code>menu_crawled</code> (고객×소스 unique, <code>menus</code> JSON·평점· <code>isPrimary</code>) · <code>menu_change_logs</code> · <code>sys_crawl_statuses</code> · <code>sys_crawl_runs</code>

중복 정리	메뉴 항목(이름+가격 키) · 검색 후보(placeId) · 소스 간(고객 1명 = 대표 메뉴 1건, 메뉴 최대 소스를 isPrimary 로)
배포	워커·서버 = git push → CodeDeploy / 다이닝코드 Lambda = CodePipeline → CodeBuild → ECR(3리전) → Lambda

운영 주의

네이버는 persistent 브라우저 프로파일 오염 시 전건 실패가 가능하다 → **프로파일 삭제 + 워커 재기동**으로 복구. 다이닝코드 Lambda는 warm 컨테이너 오염 방지를 위해 시작·종료 양쪽에서 orphan Chromium / /tmp 프로파일을 정리한다.

12.7 경쟁사 가격 크롤러

자사 상품코드(itemCode)를 입력받아 경쟁 채널(식보 · 배민상회)에서 동일/유사 상품 가격을 수집하고, 규격 기준 **환산 단가 (unitPrice)**로 정규화하여 source별 최저가를 저장한다. SKU 상태에 따라 3가지 경로로 분기한다.

분기	판정	처리
HAS_URLS	price_item_urls 에 매핑 URL 존재	매핑 URL 직접 호출 (검색·LLM 우회), 사람이 매핑 → 신뢰
NO_MATCH	매핑 행 isNoMatch=true	"경쟁사 없음": 기존 결과 정리 후 종료
NOT_TARGET	매핑 행 없음 (그 외 전체)	검색 → 후보 필터 → LLM(Gemini) 매칭

항목	내용
API / 큐	Express(:3001) · BullMQ(동시성 3) · 워커 --listen 상시
수집 기술	식보 = GraphQL 직접 호출(게스트 세션) / 배민상회 = Playwright Stealth fetch(Cloudflare WAF 우회) / 상품 매칭 = Google Gemini
가동 방식	서버 내장 스케줄러(setTimeout). 매일 02:00(KST) 전체 상품 자동 적재(토·일 제외), 04:00 스킵 로그 30일 정리, 수동 POST /admin/run-daily-cron
검색 플로우	식보·배민 병렬 수집 → regex 후보 필터(synonyms / filter-rules / excluded-sellers) → LLMMatcher 3단계 (7점 ↑ 룰베이스 PASS / 3~5점 Gemini 검증 / 0~2점 매칭 없음) → 상세 보강 → 환산 단가 → 최저가
URL 플로우	매핑 URL 수집 → 규격 분리 → 가격 편차 필터(내부가 대비 ±20% 초과 제외) → 최저가
환산 단가	$unitPrice = 외부가 \times (내부 총용량 \div 외부 총용량)$. 환산값이 원가의 50배를 벗어나면 신뢰하지 않고 원가로 대체
입력	products (atom, SKU 마스터) · price_item_urls (crawler, itemCode·source·url·isNoMatch)
출력(crawler DB)	price_crawled (itemCode×source 최신 1건, price · unitPrice · couponPrice · matchReason) · price_change_logs · price_url_skips (30일) · llm_usage_log
일배치 연계	price_crawled → 일배치 Step0가 competitor_prices 로 정규화(환산가 unit_price 우선) → 가격계획 경쟁사 2차 조정 입력(§ 19.1)

13 데이터 인터페이스 (S3 · SQS · API)

13.1 입력 데이터 (차별화상회 → ATOM, S3)

차별화상회가 매일 두 개의 S3 버킷에 데이터를 적재하면, 일배치가 파일 도착 이벤트를 받아 DB에 반영한다. 아래는 운영 (PROD) 실제 적재 기준이다.

① 상품 · 원가 버킷 `recommend-products-data-prod`

파일	업로드(KST)	크기	내용	적재 테이블
<code>product_top.json</code>	07:00	~1.4MB	매출 상위 500 상품 + AI 기본쿠폰 권장금액 (<code>ai_base_coupon_discount_amount</code>). 해당 상품에 <code>is_top_sales_product=true</code> 표시	<code>products</code> (갱신)
<code>freshon.json</code>	07:00	~9.5MB	매입원가 전체 (상품 원가 · 구매입수 · 단위)	<code>freshon_prices</code>
<code>freshon_diff.json</code>	07:00	변경분	전일 대비 원가 변경분만 (없으면 빈 파일)	<code>freshon_prices</code>
<code>product_non_top.json</code>	13:30	~23MB	상위 500 제외 일반 판매 상품. 내일 적용가(<code>reserved_*</code>) 포함	<code>products</code>
<code>product.json</code>	18:00	~24.6MB	상품 마스터 전체 스냅샷 (상품명 · 가격 · 카테고리 · 보관방식 등 48컬럼)	<code>products</code>
<code>product_diff.json</code>	18:00	변경분	전일 대비 상품 변경분만 (없으면 빈 파일, 가장 마지막에 덮어써 최신 상태 보장)	<code>products</code>

② 고객 · 주문 버킷 `recommend-customer-data-prod`

경로 `uploads/{년}/{월}/{일}/`, 고객별 개별 파일 (운영 하루 기준 접속 약 677 · 장바구니 213 · 주문 90건).

파일	업로드(KST)	내용	적재 테이블
<code>{고객ID}_access.json</code>	16:00	고객 접속 기록 (마지막 접속 시각)	<code>accesses</code>
<code>{고객ID}_cart.json</code>	16:00	장바구니 담기 내역 (상품 · 수량 · 시각)	<code>carts</code>
<code>{고객ID}_order.json</code>	16:00	주문 내역 (주문상태 · 상품 · 판매가 · 쿠폰 · 결제, 취소 포함)	<code>orders + order_items</code>
<code>customer_master.json</code>	최초 1회 + 실시간	고객 마스터 (상호 · 사업자번호 · 주소). 이후 변경은 SQS로 실시간	<code>customers</code>
<code>coupon_history.json</code> / <code>cart_coupon.json</code>	16:00경	장바구니 쿠폰 발급 이력 (전일 16시 ~ 당일 16시)	<code>coupon_histories</code>

변경분(diff) 파일 규칙

`product.json` · `freshon.json`은 전체 스냅샷, `*_diff.json`은 전일 대비 변경분만 담는다. 변경이 없으면 빈 파일(2바이트)로 올라오며, 일배치는 있을 때만 UPSERT한다. 단종 상품은 별도 삭제 파일 없이 `product_diff.json`의 상태값 변경으로 처리한다.

③ 산출물 버킷 (ATOM → 차별화상회)

버킷	내용
----	----

<code>sfn-products-suggestion-{env}</code>	추천 결과 · 상품 쿠폰 · 장바구니 쿠폰 (상세 스키마 § 13.5)
<code>sfn-recommend-lms-{env}</code>	LMS(문자) 발송 데이터 (10:00 / 15:00)

13.2 SQS 큐 (계정 099900259356)

차별화상회가 S3에 파일을 올리면 도착 이벤트가 큐로 전달되고, 일배치/API가 이를 받아 처리한다.

큐	역할	발행	소비
<code>recommend-customer-events-{env}</code>	고객 데이터 파일 도착 알림 (<code>customer_master</code> · <code>{고객ID}_order/cart/access</code>)	차별화상회 (S3)	일배치 이벤트 소비
<code>recommend-products-events-{env}</code>	상품 · 원가 파일 도착 알림 (<code>product(_non_top/_diff)</code> · <code>freshon</code>)	차별화상회 (S3)	일배치 이벤트 소비
<code>recommend-coupons-events-{env}</code>	쿠폰 이력 파일 도착 알림 (<code>coupon_history</code> · <code>cart_coupon</code>)	차별화상회 (S3)	일배치 이벤트 소비
<code>customer-recommend-modified-{env}.fifo</code>	신규/수정 고객 실시간 이벤트 (가입 · 정보변경 즉시, 순서보장 FIFO)	차별화상회 (직접 발행)	API SQS 소비

※ 위 4개가 실제 사용 중인 큐다. 이 외에 과거 설계 흔적으로 프로비저닝만 되어 있고 현재 사용하지 않는 큐가 2개 더 존재하나, 운영에는 관여하지 않는다.

앞 3개(events)와 .fifo의 차이

events 큐는 "파일이 올라왔다"는 도착 알림만 담는다. 일배치가 알림을 받아 해당 파일을 S3에서 읽어 DB에 적재한다. **.fifo** 큐는 신규/수정 고객 정보 자체를 담은 실시간 이벤트로, API가 즉시 처리해 신규 고객에게 바로 추천을 만든다.

13.3 차별화상회 발급/등록 API (ATOM → 차별화상회, Basic Auth)

배치가 core/http_client.py(외부 API 클라이언트)로 호출. 베이스 URL은 API_BASE_URL(PROD http://api.iprod.chabyulhwa.local/external/partner), 인증은 Basic Auth(자격증명 마스크·부록 C).

엔드포인트	용도	단계
POST /coupons/base/create	기본쿠폰 생성 (전 상품, AI_BASE)	Phase 2
POST /coupons/ai-rec/create	AI 추천쿠폰 스킴 생성 (PRODUCT/CART, AI_REC) → meta_coupon_number 반환	Phase 3
POST /coupons/ai-rec/issue	스킴별 대상 고객 발급	Phase 3
가격 등록 API	planned_prices 가격/정가 등록 (batch 500)	Phase 2 (Step5)

13.4 ATOM API 엔드포인트 (FastAPI · ATOM Engine)

엔드포인트	용도
POST /api/v1/profiling/save	프로파일링 서비스 → 식자재 추출 결과를 차별화상회 RDS에 저장 (VPC 피어링, 다건 batch)
POST /api/webhook/crawl-complete	크롤링 서버 완료 콜백 (type=menu price) → 후속 파이프라인 트리거
POST /api/webhook/customer-event	SQS 고객 이벤트 HTTP 보조 수신
GET /health · /dicts/* · /admin/*	헬스체크 · 검색사전 · 관리(승인 상태 조회 등)

승인 게이트

배치 Phase 2는 GET {ATOM_API}/api/v1/approvals/{batch_date}/status를 5분 폴링하여 승인 결과를 확인한다(대시보드 백엔드 제공). INSPECTION_WORKFLOW_ENABLED=false이면 자동 통과.

프로파일링 서비스의 DB 격리

프로파일링 서비스는 **DB에 직접 접근하지 않는다**. 모든 결과는 위 /api/v1/profiling/save를 통해서만 차별화상회 RDS에 기록된다(절대 규칙).

13.5 S3 산출물 스키마 & 예시 (ATOM → 차별화상회)

일배치 Phase 3에서 **유저별 추천**(`{user_id}_recommend.json`) · **상품 쿠폰**(`coupon.json`) · **장바구니 쿠폰**(`cart_coupon.json`)을 S3에 적재하면 차별화상회가 이벤트로 감지해 노출한다. 포맷 규약: JSON · UTF-8 · 시간 ISO 8601(KST) · 금액 원 단위 정수.

① 추천 결과: {user_id}_recommend.json

```
{
  "user_id": 12345,
  "transaction_id": null,          // SQS 동기화 요청 건만 값 존재, 그 외 null
  "items": [
    { "product_id": 100123, "score": 0.8742, "recommend_type": "REPURCHASE",
      "is_lowest_sale_price": true, "is_lowest_coupon_price": true,
      "is_imminent": true, "imminent_days": -2 },
    { "product_id": 100456, "score": 0.6531, "recommend_type": "RECOMMENDATION",
      "is_lowest_sale_price": false, "is_lowest_coupon_price": true,
      "is_imminent": false, "imminent_days": null }
  ]
}
```

필드	설명
recommend_type	REPURCHASE(재구매: 30일내 신선/60일내 가공) / RECOMMENDATION(상품 추천)
is_lowest_sale_price	최저가 여부: 판매가 기준(판매가 = min(정가, 캠페인할인가))
is_lowest_coupon_price	최저가 여부: 상품쿠폰 포함 기준
is_imminent / imminent_days	주문임박(재구매 전용) / 0=당일, 양수=N일 후, 음수=N일 전, null=데이터 없음

② 상품 쿠폰 배정: coupon.json

```
{
  "user_id": 12345,
  "coupons": [
    { "product_id": 100123, "coupon_amount": 800, "coupon_type": "DYNAMIC_PRICING" },
    { "product_id": 100456, "coupon_amount": 1500, "coupon_type": "RECOMMENDATION" }
  ]
}
```

coupon_type (3종): DYNAMIC_PRICING(재구매·4-Tier) / RECOMMENDATION(미구매·메뉴기반 식자재 차등) / CHURN_PREVENTION(이탈·B/S 차등). 분기: 이탈 고객 → CHURN_PREVENTION, 재구매 → DYNAMIC_PRICING, 그 외 → RECOMMENDATION.

※ 기본쿠폰(BASE)은 coupon.json에 포함되지 않는다. 기본쿠폰은 별도 API POST /coupons/base/create로 발급되며, 구분자는 coupon_type이 아니라 issue_source_type = AI_BASE다.

③ 장바구니 쿠폰 배정: cart_coupon.json

```
{ "user_id": 12345, "coupon_amount": 8000, "min_order_amount": 150000 }
// coupon_amount = 0 이면 해당 고객에게 장바구니 쿠폰 미발급
```

④ Up-price 가격 흐름 (예시)

적용 단계	정가	판매가	쿠폰	고객 결제가
원본가 (차별화상회 전달)	10,000	10,000	-	-
Up-price +10% 적용	11,000	11,000	-	-
기본쿠폰 (비로그인·전 고객)	11,000	11,000	-1,000	10,000 (원본가 유지)

개인화쿠폰 (구매이력 고객, 4-Tier)	11,000	11,000	-800	10,200
개인화쿠폰 (미구매 + 식자재 매칭)	11,000	11,000	-1,500	9,500 (첫 구매 유도)

※ 금액은 예시. 인상율은 상품별 랜덤 10~15%, 쿠폰액은 모델 산출 결과에 따라 달라진다.

14 DB 스키마: ERD (핵심 테이블 관계)

전체 74개 테이블 중 핵심 관계를 먼저 보인다. 데이터는 **원천(차별화상회 제공)**과 **산출(Weraser 생성)**으로 나뉘며, 일배치의 모든 모델은 customer_product_facts(고객×상품 통합 팩트)로 모인다.

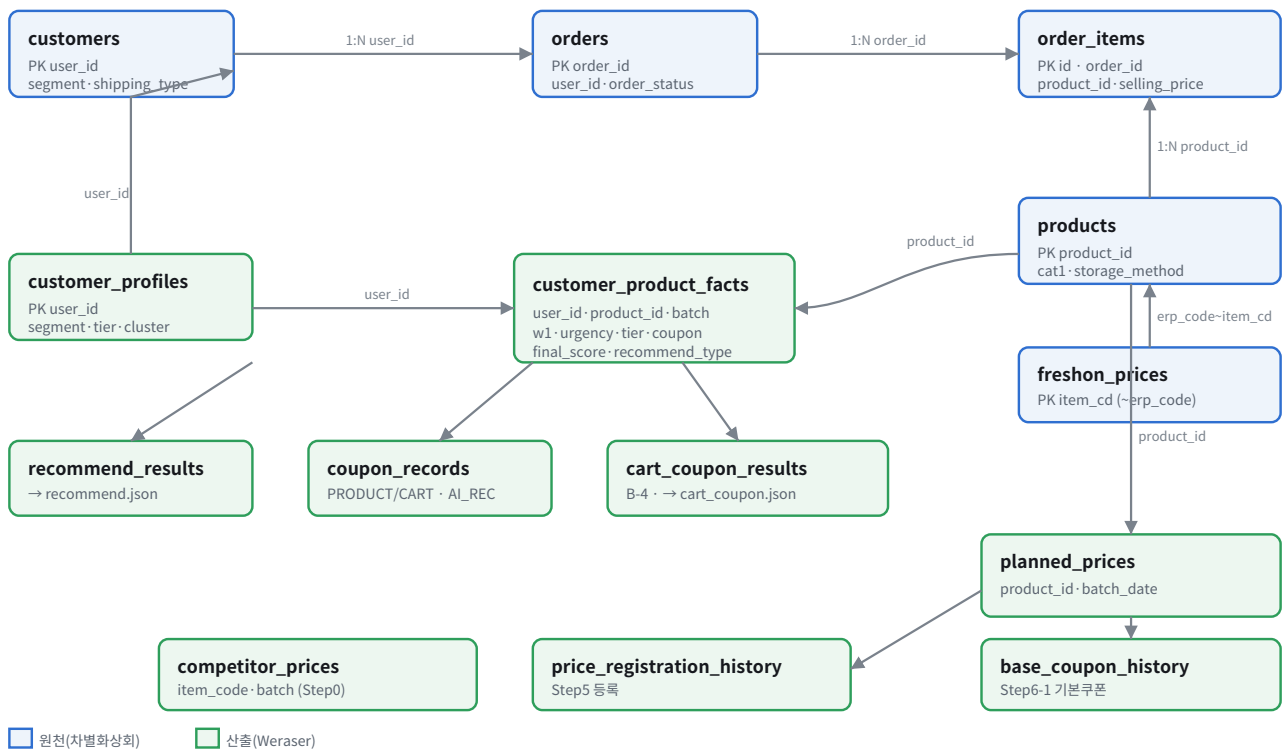


그림 14-1. 핵심 ERD. 관계는 논리적 흐름 기준(물리 FK는 일부만, 대부분 raw SQL 조인). freshon_prices는 products.erp_code ↔ item_cd, competitor_prices는 products.product_id ↔ item_code 매핑.

구분	주요 테이블
원천 (차별화상회)	customers · products · freshon_prices · orders · order_items · carts · accesses · coupon_histories · fc_headquarters · fc_customers · product_sku_mapping
산출 (Weraser)	customer_profiles · customer_product_facts · recommend_results · coupon_records · planned_prices · cart_coupon_results · competitor_prices · base_coupon_history · price_registration_history · profiling_* · sku_mapping_history 등

분리 원칙

ATOM은 원천 데이터를 수정하지 않는다. 원천은 차별화상회가 S3/SQS로 적재한 그대로 읽기만 하고, 모든 분석 결과는 산출 테이블에만 기록한다. 물리 외래키(FK)는 일부만 걸려 있고 대부분 raw SQL 조인으로 연결된다.

15 원천 데이터 테이블 (차별화상회)

차별화상회가 제공한 데이터를 그대로 적재한다. 컬럼은 운영 DB 기준이며, 각 테이블의 인덱스를 함께 표기한다.

15.1 customers 고객 마스터 · PK user_id

컬럼	타입	설명
user_id	bigint	고객 ID (PK)
company_name	varchar(255)	상호(매장명)
business_registration_no	varchar(20)	사업자등록번호
zip_code / address / address_detail	varchar / text	우편번호 · 기본주소 · 상세주소
business_type	varchar(50)	업종 (차별화상회 11개 분류)
customer_type	varchar(30)	NORMAL / LEGACY_SUPER_BUSINESS 등
shipping_type	varchar(20)	DIRECT(직배송) / PARCEL(택배) . 장바구니·이탈 쿠폰·LMS는 DIRECT 대상
erp_code	varchar(50)	ERP 코드
last_order_at / last_accessed_at	timestamp	최종 주문 · 최종 접속
is_active / purchase_status / is_tester	bool / varchar	활성 여부 · 구매 상태 · 테스터 계정
fc_brand_name	varchar(100)	프랜차이즈 브랜드명

인덱스 · PK(user_id) · idx_customers_fc_brand(fc_brand_name) [부분: fc_brand_name IS NOT NULL]

15.2 products 상품 마스터 · PK product_id · 총 54컬럼

컬럼	타입	설명
product_id	integer	상품 ID (PK)
name / full_name / description	varchar / text	상품명 · 전체명 · 설명
purchase_price	integer	매입원가
status	varchar(30)	ACTIVE / INACTIVE / TEMPORARILY_SOLD_OUT
product_type / tax_type / delivery_type	varchar	상품·과세·배송 유형
storage_method	varchar(20)	REFRIGERATION / FREEZING / ROOM_TEMPERATURE (B-3 신선/가공 분류)
list_price / price / discount_rate	integer	정가 · 판매가 · 할인율
reserved_price_start_date	varchar(30)	예약가 적용 시작일 (보통 익일)
reserved_list_price / reserved_discount_price	integer	내일 적용 정가 · 판매가 (일반상품은 product_non_top.json으로 적재)
erp_code / erp_name	varchar	ERP 코드(↔ freshon item_cd) · ERP 명

sale_unit / quantity_of_uom / unit_of_measure	varchar / double	판매 단위 · 단위당 수량 · 측정 단위 (환산용)
leaf_category_id / leaf_category_name / cat1~cat4	integer / varchar	리프 카테고리 · 4단 분류
bundle_sale_yn / bundle_quantity	bool / integer	묶음 판매 여부 · 묶음 수량
coupon_applicable_status	varchar(20)	쿠폰 발급 가능 여부
price_modification_applicable_status	varchar(20)	동적 프라이싱 가격 수정 가능 여부
recommendation_boost_required	bool	추천 가중치 부스트 대상
is_top_sales_product	bool	매출 상위 500 상품 여부 (product_top.json로 매일 갱신)
ai_base_coupon_discount_amount	integer	AI 권장 기본쿠폰 금액 (상위 500)

인덱스 · PK(product_id) · idx_products_top_sales(is_top_sales_product) [부분: = true]

15.3 freshon_prices 매입원가 · PK item_cd

컬럼	타입	설명
item_cd	varchar(50)	원가 상품 코드 (PK, ↔ products.erp_code)
buy_price	double	매입원가 (GP 마진 산출 기준)
buy_rate	double	구매 입수율
stock_unit / order_unit	varchar(10)	재고 단위 · 발주 단위
start_date / use_yn	varchar	판매 시작일(yyyyMMdd) · 사용 여부(Y/N)

인덱스 · PK(item_cd)

15.4 orders 주문 · PK order_id

컬럼	타입	설명
order_id	varchar(30)	주문 ID (PK)
user_id	bigint	고객 ID
ordered_at	timestamp	주문일시
order_status	varchar(20)	PENDING / PAYMENT / CANCEL_COMPLETED / PARTIAL_CANCELED

인덱스 · PK(order_id) · ix_orders_ordered_at(ordered_at) · ix_orders_user_id(user_id)

15.5 order_items 주문 상세 · PK id

컬럼	타입	설명
id	integer	주문상세 ID (PK)
order_id / product_id	varchar / integer	주문 ID · 상품 ID

quantity / selling_price / order_amount	integer	수량 · 판매단가 · 주문금액
supply_price	integer	공급가
product_coupon_discount (+_supply_price)	integer	상품쿠폰 할인액 (공급가 기준 동반)
cart_coupon_allocated (+_supply_price)	integer	장바구니쿠폰 배분 할인액 (체리피커 산출 기준)
used_point_amount / pg_payment_amount (+_supply)	integer	포인트 사용 · PG 결제액
product_coupon_code / cart_coupon_code	varchar(100)	적용된 상품·장바구니 쿠폰 번호 (스킴 ID)

인덱스 · PK(id) · ix_order_items_order_id(order_id) · ix_order_items_product_id(product_id)

16 산출 데이터 테이블 (Weraser)

ATOM이 계산해 생성하는 데이터다. 핵심 7개 테이블을 구조·인덱스와 함께 싣고, 나머지는 카탈로그로 정리한다.

16.1 customer_profiles 고객 분석 프로필 · PK user_id · user당 1행

컬럼	설명
user_id	고객 ID (PK)
segment	A-1 세그먼트 (자사 FC / 타사 FC / 일반식당)
customer_tier	A-2 티어 (이탈-위험 / 이탈-확실 / 이탈-확실(장기) / 잠재(미구매) 등)
is_churned	이탈 여부
basket_size / monthly_orders	평균 주문금액(B/S) · 월 구매 빈도
is_cherry_picker / coupon_response_type / coupon_usage_rate	A-3 체리피커 · 쿠폰 반응 유형 · 쿠폰 사용률
primary_cluster_id / sub_cluster_id / silhouette_score	B-1 서브클러스터링 결과
matched_brand	매칭된 프랜차이즈 브랜드
batch_date	배치 기준일

인덱스 · PK(user_id) · idx_cp_primary_cluster(primary_cluster_id) · ix_customer_profiles_batch_date(batch_date)

16.2 customer_product_facts 고객×상품 통합 팩트 · PK id · 당일치만 보관 · 37컬럼

핵심 테이블

모든 모델(A·B·C)의 산출이 이 테이블 한 곳에 합쳐진다. 매 Phase 3 종료 시 당일 외 데이터를 삭제(batch_date < 당일)하므로 당일치만 유지된다. 복구는 재산출 또는 RDS PITR.

컬럼	설명 (모델)
user_id / product_id / batch_date	논리 유니크 키 (고객×상품×기준일)
purchase_count / total_quantity / total_amount	주문 집계

first_purchase_at / last_purchase_at / avg_purchase_interval_days	구매일 · 평균 간격
coupon_usage_count / coupon_total_discount	쿠폰 사용 통계
revenue_share / price_tier	A-5 매출 비중(0~1) / TOP · KEY · STANDARD · TAIL
w1_score	B-2 연관구매 점수(0~1)
d_day / urgency_score / pavg_days / oavg_days / is_imminent	B-3 구매주기 · 긴급도 · 간격 · 임박
coupon_amount	A-4 상품 쿠폰 금액
ingredient_freq_score / profiling_ingredient_count	C-3 식자재 빈도 점수 · 매칭 식자재 수
crawl_source / crawl_place_name / crawl_menus_json	크롤링 출처 · 매장명 · 메뉴(JSON)
segment / business_type / storage_method / primary_cluster_id / sub_cluster_id	세그먼트 · 업종 · 보관방식 · 클러스터 (조인 캐시)
final_score / recommend_type	A-6 최종 스코어 · REPURCHASE/ RECOMMENDATION

인덱스 · PK(id) · 논리 유니크(user_id, product_id, batch_date) · idx_cpf_ingredient_freq(ingredient_freq_score, 부분) ·
idx_facts_primary_cluster · ix_*_batch_date · ix_*_product_id · ix_*_user_id

16.3 planned_prices 가격계획 · PK (product_id, batch_date)

컬럼	설명
product_id / batch_date	복합 PK
original_list_price / original_price	원본 정가 · 판매가
new_list_price / new_price / markup_rate	Up-price 적용 정가 · 판매가 · 인상률
buy_price	매입원가
base_coupon_amount / base_coupon_source	기본쿠폰 금액 · 산출 출처(지정가/원가기반/당일혜택가/경쟁사/묶음/마스터유 지/없음)
new_benefit_price / price_source / coupon_applicable	기본혜택가 · 가격 출처 · 쿠폰 적용 가능

인덱스 · PK(product_id, batch_date)

16.4 recommend_results 추천 결과 · PK id · → recommend.json

컬럼	설명
id	PK
user_id / product_id	고객 · 추천 상품
score	추천 점수 numeric(10,6)
recommend_type	REPURCHASE / RECOMMENDATION
is_lowest_sale_price / is_lowest_coupon_price	판매가 기준 · 상품쿠폰 포함 기준 최저가 여부
is_imminent / imminent_days	주문임박 · 임박일(0/양수/음수/null)

transaction_id / batch_date	무결성 ID(SQS 건) · 기준일
-----------------------------	---------------------

인덱스 · PK(id) · ix_recommend_results_batch_date · ix_recommend_results_user_id

16.5 coupon_records 쿠폰 발급 이력 · PK id

컬럼	설명
id	PK
user_id / product_id	대상 고객 · 상품
meta_coupon_number	쿠폰 스킴 번호 (발급 식별자)
coupon_type	PRODUCT / CART
issue_source_type	AI_BASE / AI_REC
discount_value / status	할인액 · 상태(CREATED/ISSUED)
batch_date / issued_at	기준일 · 발급 시각

인덱스 · PK(id) · ix_coupon_records_batch_date · ix_coupon_records_meta_coupon_number · ix_coupon_records_user_id

16.6 cart_coupon_results 장바구니 쿠폰(B-4) · PK id · → cart_coupon.json

컬럼	설명
id / user_id	PK · 고객
coupon_amount / min_order_amount	후처리 적용 최종 할인액 · 최소주문금액
order_count / q1 / q2 / q3	B-4 분위회귀 입력 (주문수 · 사분위)
additional_purchase / personal_gp_rate / bep_amount	추가구매 유도분 · 개인 GP율 · 손익분기
raw_coupon_amount / raw_min_order_amount	후처리 전 원본 값
coupon_amount_gp100 / gp120 / gp140	GP 시나리오별 금액 (모니터링)
coupon_type / is_vip_coupon / coupon_issue_weekday / order_hour_band	유형 · VIP 여부 · 발급 요일 · 발송 시간대
data_source / batch_date	산출 근거 · 기준일

인덱스 · PK(id) · ix_cart_coupon_results_batch_date · ix_cart_coupon_results_user_id

16.7 competitor_prices 경쟁사 최저가 · PK (item_code, batch_date)

컬럼	설명
item_code / batch_date	복합 PK (↔ products.product_id)
lowest_price / lowest_source	최저가 · 출처(식품/배민)
lowest_unit_price / lowest_unit_source	환산 단가 최저가 · 출처 (가격계획 2차조정 기준)
crawl_count	수집 건수

인덱스 · PK(item_code, batch_date)

16.8 그 외 산출 테이블 (카탈로그)

테이블	설명
base_coupon_history	기본쿠폰 생성 이력 (Step6-1)
price_registration_history	가격 등록 이력 (Step5, SUCCESS/FAILED)
customer_menus / customer_industry / customer_send_pattern	크롤링 메뉴 · 업종 폴백 · LMS 발송 패턴
profiling_results / profiling_menu_items / profiling_ingredients	C-1 메뉴 요약 / 메뉴별 식자재 / 고객 최종 식자재
sku_mapping_history / sku_llm_verifications	C-3 SKU 매핑(출처 레이어) / Gemini 의미검증 캐시
industry_popular_products / cluster_popular_products / fc_brand_recommended_products	업종·클러스터·FC 브랜드 인기 상품 (폴백 폴)
lms_message_logs / lms_marketing_messages / lms_suppression	LMS 발송 이력 / 마케팅 메시지 / 발송 억제
batch_runs / batch_progress_logs / batch_approvals / event_logs	배치 실행 · 단계 진행률 · 승인 · 이벤트 수신 기록
customer_pipeline_status / processed_s3_events / sqs_dead_letters	고객 파이프라인 상태 · 처리한 S3 이벤트 · DLQ

17 모델 상세 (A · B · C)

ATOM의 추천·쿠폰은 3계열 모델로 산출된다. **A** 자체 비즈니스 로직(배치 recommendation/), **B** AI 모델 라이브러리(b_models), **C** 프로파일링(프로파일링 서비스 + sku_mapper).

17.1 모델 카탈로그

모델	위치	출력	실행
A-1 세그먼테이션	recommendation/segmentation.py	segment, matched_brand	P3 Step4
A-2 이탈·티어	recommendation/churn_detector.py	customer_tier, is_churned	P3 Step4
A-3 쿠폰반응	recommendation/coupon_response.py	is_cherry_picker	P3 Step4
A-4 상품쿠폰(재구매)	recommendation/product_coupon.py	coupon_amount	P3 Step4
A-4b 미구매 추천쿠폰	step4_a_models._run_a4b_unpurchased_coupon	coupon_amount(RECOMMENDATION)	P3 Step4
A-5 가격민감도	recommendation/price_sensitivity.py	price_tier (매출순 4등분)	P3 Step4

A-6 가중합	recommendation/scorer.py	final_score, recommend_type	P3 Step4
B-1 서브클러스터	b_models/b1_subclustering	primary/sub_cluster_id, silhouette	P3 Step3
B-2 연관구매	b_models/b2_association	w1_score	P3 Step3
B-3 구매주기	b_models/b3_purchase_cycle	d_day, urgency_score, pavg/oavg_days	P3 Step3
B-4 장바구니쿠폰	b_models/b4_quantile	coupon_amount, min_order, weekday	P3 Step3
C-1 메뉴매칭	프로파일링 서비스	profiling_ingredients	P1
C-2 업종매핑	프로파일링 서비스	business_type	P1
C-3 SKU매핑(4-Layer)	recommendation/sku_mapper	facts RECOMMENDATION 행	P1

17.2 A-6 최종 스코어 (가중합)

일배치는 customer_product_facts에 모인 모든 모델 결과를 읽어 고객×상품마다 최종 추천 점수를 매긴다. 절차는 다섯 단계다.

- 분류:** 구매 이력이 있고(구매횟수>0) 최근 구매가 신선 30일 / 가공 60일 이내면 **재구매(REPURCHASE)**, 그 외에는 **신규(RECOMMENDATION)**로 나눈다.
- 정규화:** 각 지표를 0~1로 맞춘다. 연관구매 점수·식자재 빈도는 그대로(0~1), 구매실적은 로그 환산($\min(1, \text{구매횟수}/10)$), 긴급도·클러스터·업종인기는 최소-최대 정규화. 혜택률 = (판매가 - 원가) / 판매가, 가격갭 = (경쟁사가 - 자사가) / 경쟁사가.
- 기본 점수(가중합):** 풀별로 아래 가중치를 곱해 더한다.

지표	재구매	신규	의미
연관구매(w1)	0.30	0.30	함께 사는 경향 (B-2)
구매주기 긴급도	0.25	-	다시 살 때가 됐는지 (B-3)
구매실적	0.30	-	그동안 얼마나 샀는지
가격민감도 티어	0.15	-	TOP 1.0 / KEY 0.75 / STANDARD 0.5 / TAIL 0.25 (A-5)
식자재 빈도	0.15	0.15	매장 메뉴에 필요한 식자재 (C-3)
혜택률(GP)	-	0.10	마진 여력
가격갭	-	0.10	경쟁사 대비 우위
클러스터 친화도	-	0.15	비슷한 고객군 선호 (B-1)
업종 인기(폴백)	0.20		구매 이력이 없을 때 업종 인기 상품으로 보완

- 보정(부스트·페널티):** 기본 점수에 상황별 가감을 더한다.

보정	값	적용 조건
긴급 재구매	+1.0	30일 이상 미구매 + 주기 초과 (최우선 노출)

구매 임박 근접	+0.5~1.0	다시 살 시점이 가까울수록 가산
상품 이탈 방지	+0~0.15	평소보다 안 사는 상품일수록 가산 (이탈비 2.0~4.0배)
이탈 고객 부스트	+0.10	이탈 위험 고객
추천 부스트 / 판매상위 부스트	+0.10 / +0.05	추천 강화 상품 / 매출 상위 상품
체리피커 패널티	-0.05	쿠폰만 골라 쓰는 고객

5. 최종 점수: 기본 점수와 보정을 합한 뒤 0 미만은 0으로 막는다. 이 점수 순으로 추천이 노출된다.

정책 메모

과거에 임박 상품에 +1000을 더해 무조건 최상단에 올리던 방식은 2026-05-14에 폐지하고(임박 부스트 0), 위의 근접 가산으로 대체했다. 재구매 판정에는 6개월(180일) 상한과 신선 30일/가공 60일 초과 행 제거, 미판매·동일 카테고리 대체 상품 제외 가드가 함께 적용된다.

17.3 A-5 가격민감도 4-Tier

최근 **6개월 내 2회 이상** 구매한 상품을 고객별로 묶어, **매출 비중이 큰 순서**로 줄을 세운 뒤 넷으로 나눈다. 상위 25%는 **TOP**, 다음 25%는 **KEY**, 그다음은 **STANDARD**, 나머지는 **TAIL**이며 매출 1위 상품은 항상 TOP이다. 이 티어로 쿠폰 할인 폭 (TOP·KEY는 크게·경쟁사가 5% 상한, STANDARD·TAIL은 작게·10% 상한)과 추천 점수 가산을 차등한다.

2026-06-19 정정

이전에는 누적 매출비중 0.25/0.5/0.75로 끊어, 매출이 한두 상품에 쏠린 고객은 1위 상품이 구간을 건너뛰며 TOP·KEY가 비고 TAIL이 과다해지는 문제가 있었다. 정의서의 "상위 25%" 취지에 맞춰 **순위 4등분**으로 바로잡았다.

17.4 A-4 / A-4b 상품 쿠폰

A-4(재구매 쿠폰)는 6개월 내 2회 이상 산 상품에 티어별 차등 할인을 준다. **A-4b(미구매 추천쿠폰)**는 아직 안 산 상품(구매횟수 0)에 주는 쿠폰으로(2026-06-19 차별화상회 확정), 다음과 같이 정한다.

- 할인액 = 기본혜택가의 5%와 (기본혜택가 - 원가) 중 작은 값. 음수면 0, 최소 1,000원.
- 경쟁사 겹은 쓰지 않고 **역마진만 방지**한다(원가 한도). 원가 정보가 없으면 GP율 21.7%를 가정한다.
- 구매횟수 0 조건으로 재구매 쿠폰(A-4)과 분리되고, A-6가 신규(RECOMMENDATION) 타입과 점수를 부여해 Step6에서 발급된다. 산출물에는 coupon_type=RECOMMENDATION으로 나간다.

17.5 B 모델 (AI 모델 라이브러리 · b_models: I/O 계약: b_models/docs/io_spec.md)

- **B-1 서브클러스터링:** facts + 업종 + 크롤링 메뉴가격 → {업종}_{가격대}_비FC 1차 클러스터 + 서브클러스터(SVD → K-Means) + silhouette. 클러스터 수는 고정값이 아니라 데이터 크기·실루엣 점수로 동적 결정(데이터가 적으면 3~5개로 상한 제한). FC 사전 제외. 산출 primary_cluster_id, sub_cluster_id, customer_type.
- **B-2 연관구매:** 주문 동시구매 support/confidence/lift → w1_normalized(0~1). 기구매 상품(owned) 후보 제외 → REPURCHASE 풀 실데이터상 w1≈0. 클러스터 인지 4단계 풀백으로 후보별 첫 매칭 단계를 채택: cluster_sku(고객 클러스터 내 SKU 규칙) → cluster_leaf(같은 클러스터, 리프 단위) → all_sku(전체 SKU 규칙) → all_leaf(전체 리프 단위). 채택 단계는 score_source에 기록.
- **B-3 구매주기**(점수 정의서 2026-03-17): 6개월(180일) cutoff. P_avg=Median(개별 구매간격). 데이터가 부족한 상품은 6단계 Cold Start 풀백(앞 단계 우선): individual(개인 2회+) → product_pavg(같은 상품 재구매 고객 30명 이상의 P_avg 중앙값) → leaf_pavg(같은 리프 30명 이상) → leaf_sibling_pavg(리프+형제 30명 이상) → freshness_default(신선 7일 / 가공·비품 30

일) → global(전역 기본 10일). 산출 d_day, urgency_score, pavg/oavg_days, 채택 단계는 data_source에 기록. 임박 윈도우 W_dynamic=Clamp(0_avg, 3, 7).

- **B-4 장바구니쿠폰(분위회귀):** additional_purchase=max(Q3-Q2, 0). **운영 쿠폰 원안 = additional_purchase × 0.8** → 절대 한도(3,000~50,000원)·최소주문 대비 할인율 3.5~6.8% 범위로 clip → 반올림. min_order = (additional_purchase < 50,000 이면 Q3+50,000, 아니면 Q3)을 1,000원 단위 반올림 후 [15만~50만] clip. 폴백: 주문 3건 미만 → 10,000/200,000원.

BEP=(additional_purchase × personal_gp_rate)는 발급에 쓰이지 않고 검수 비교 컬럼 gp100/120/140(BEP × 1.0/1.2/1.4) 전용. 최종 발급 금액은 배치 오케스트레이터 후처리(§ 19.3)가 정본.

- **fc_matching:** FC(프랜차이즈) 브랜드 매칭 모듈(b_models/fc_matching). 자사/타사 FC 고객을 브랜드에 매칭하고 (matched_brand), 브랜드 추천 상품(fc_brand_recommended_products) 풀을 추천에 활용한다. 서브클러스터링(B-1)에서는 FC를 사전 제외하고 이 경로로 별도 처리한다.

17.6 C 모델 (프로파일링)

- **C-1 메뉴매칭:** 크롤링 메뉴 → 사전매칭(exact/fuzzy)/LLM 추출 → profiling_ingredients. 프로파일링 서비스(Gemini)가 담당, ATOM은 /api/v1/profiling/save로 결과 수신(DB 직접접근 금지).
- **C-2 업종매핑:** 크롤링 카테고리 → 차별화상회 11개 업종(business_type) 폴백.
- **C-3 SKU매핑(4-Layer):** 식자재→상품(SKU). 우선순위 sf_n_brand(자사 FC 정답지) → sf_n_name(SKU 매핑 마스터) → personal(개인) → industry(업종 인기). 하위 매칭 엔진은 dami_matcher(식자재명 유사도 매칭) · ingredient_matcher(식자재 →SKU 후보 선정) · sku_llm_verifier(점수 0.5 미만 시 Gemini 의미검증, sku_llm_verifications 캐시)로 구성된다. 결과 → facts RECOMMENDATION 행 + ingredient_freq_score(w3).

18 배치 전체 로직 (3-Phase + 스케줄러)

운영 배치는 atom-scheduler 서비스(workers/scheduler.py, v2.1)가 **30초 폴링 루프**로 시각을 감시하며 Phase를 트리거한다. 각 작업은 executed_today 집합으로 **하루 1회**만 실행된다.



※ 주말: Phase 1 가격계획·LMS·검수 스킵(금요일 가격 유지), Phase 2는 기본쿠폰만(승인 불필요), Phase 3

그림 18-1. v2.1 일배치 타임라인(평일/KST). 모델·개인화/장바구니 쿠폰은 18:00 최신 마스터 도착 후 Phase 3에서 산출.

18.1 Phase별 단계

Phase	단계	비고
Phase 1 08:00	Step1 S3적재(product_top·freshon) → Step1-2 차별화상회 적용가 드리프트 감지(평일) → Step0 경쟁사가(평일) → 가격계획 (상위 500, is_top_sales_product ; 평일) → Step2 팩트빌드 → Step2-init C-3 사전갱신 → Step2-1 C-3 SKU매핑 → Step2-2 팩트보강(enrich, fresh 세션)	가격 산출에 집중. B/A 모델은 Phase 3로 이동 (v2.1). 팩트는 전일까지 DB 데이터 사용
LMS AM 10:00	step_lms 오전 발송	평일. D-1 산출 기반(Phase 1 완료 비의존). DB 이력으로 중복방지

Report 14:00	Step8 검수 리포트 메일 + 승인 토큰 발급	평일만. INSPECTION_WORKFLOW_ENABLED 로 제어
LMS PM 15:00	step_lms 오후 발송	평일
Phase 2 16:10	승인 확인(폴링) → Step5 가격등록(상위 500) + Step6-1 기본쿠폰 생성	Phase 1 task 완료 대기. 주말은 기본쿠폰만(승인 불필요)
Phase 3 18:10	Step3 B모델(B-2·B-3·B-4) → Step4 A모델(A-2~A-6) → Step6-2 개인화쿠폰 (PRODUCT) 발급 → Step6-3 장바구니쿠폰(CART) 발급 → Step7 S3 업로드 → 팩트 클린업	18:00 product.json/diff 도착 후. Phase 2 미 완료 시 스킵

18.2 승인 게이트

평일 Phase 2 진입 시 GET {ATOM_API}/api/v1/approvals/{batch_date}/status를 **5분 간격 폴링**한다.

- **approved** → Phase 2 실행 + phase2_done 표시 → Phase 3 진행 가능
- **rejected / 17:30 마감 타임아웃** → Phase 2 스킵 → Phase 3도 스킵(가격등록 선행 필요)
- INSPECTION_WORKFLOW_ENABLED=false → 폴링 없이 자동 통과(단 INTEGRATION_ENABLED 가드가 외부 호출은 차단)

18.3 스케줄러 신뢰성

메커니즘	동작
30초 폴링 루프	매 30초 현재 시각 체크, 시각 도달 시 Phase 트리거(background task)
중복 실행 방지	executed_today 집합: 하루 1회. 날짜 변경 시 초기화
재시작 catch-up 스킵	시작 시 이미 지난 시각 작업은 스킵(재실행 방지). 시각 이전 재시작은 정상 실행
Phase 2 완료 복원	재시작 시 price_registration_history SUCCESS 확인 → phase2_done 복원(Phase 3 오스킵 방지)
LMS 중복 방지	lms_message_logs DB 이력(lms_already_sent_today) 확인
지연 알림	Phase 1 > 2h, Phase 3 > 3h 초과 시 Slack @channel 멘션
Admin 서버	스케줄러와 함께 :8200 동시 기동

18.4 보조 프로세스

- **atom-s3-consumer**: S3 이벤트(SQS 3큐) 실시간 소비 → 원천 데이터 DB 적재. 배치와 독립.
- **atom-sqs**(API측): 신규/수정 고객 FIFO 큐 소비 → 즉시 추천 처리.
- **팩트 보존**: Phase 3 종료 시 DELETE FROM customer_product_facts WHERE batch_date < 당일: 당일치기만 유지.

18.5 상시 처리 워커 (일배치와 별개 경로)

아래 두 워커는 일배치 스케줄과 무관하게 API 서버에서 24시간 상시 구동된다. 일배치가 "하루 한 번 전체"를 처리한다면, 이 워커들은 "수시로 들어오는 신규 건"을 즉시 처리한다.

신규·수정 고객 실시간 추천 (SQS 소비기)

차별화상회가 고객을 신규 가입·정보 변경하면 `customer-recommend-modified-{env}.fifo` 큐로 CREATE/UPDATE/DELETE 이벤트가 들어오고, API SQS 소비기가 이를 받아 **해당 고객만 즉시 추천 산출한다**(일배치를 기다리지 않음). 산출 로직은 일배치와 동일한 C-3 경로를 탄다.

항목	동작
롱 폴링	최대 10건씩 20초 long polling으로 수신
가시성 · 하트비트	수신 즉시 visibility timeout 동안 다른 워커에 보이지 않고, 처리가 길어지면 240초마다 visibility를 300초로 연장(heartbeat)해 중복 처리를 막는다
성공 / 실패	성공 시 메시지 삭제, 실패 시 삭제하지 않아 timeout 후 SQS가 재전달
DLQ	수신 5회 초과 시 강제 삭제하고 <code>sqs_dead_letters</code> 테이블에 보관(원인 추적). 모든 수신은 <code>event_logs</code> 에 RECEIVED / SUCCESS / FAILED / DEAD_LETTER로 기록
FIFO 순서	MessageGroupId 단위로 순서 보장. 같은 그룹은 앞 메시지를 처리·삭제한 뒤 다음 메시지를 전달(앞 건 실패 시 그룹이 timeout까지 대기)
종료	SIGTERM 수신 시 처리 중인 메시지를 마치고 안전 종료(graceful)

프로파일링 워커 (24시간 식자재 추출, C-1)

API 서버의 프로파일링 워커가 상시 구동하며, 아직 식자재가 추출되지 않은 고객을 순회해 **크롤링 → 메뉴 매칭 → 식자재 추출 → 저장**을 수행한다. 크롤링 메뉴가 있고(menu_crawled success) profiling_results에 없는 활성 고객을 10명씩 묶어 위레이저 프로파일링 API로 처리하고(배치 간 30초 대기), 미처리 고객이 없으면 300초 대기 후 재스캔해 **신규 가입자를 자동으로 다시 처리**한다. 결과는 API(/api/v1/profiling/save)를 통해서만 RDS에 저장된다.

19 가격 · 쿠폰 정책

19.1 동적 프라이싱(Up-price): step_price_plan.py

Phase 1에서 planned_prices를 산출(상위 판매상품 is_top_sales_product 중심 + 예약가/AI쿠폰 보유/묶음 멤버 포함), Phase 2 Step5에서 승인 후 차별화상회에 등록한다.

동적 프라이싱은 **기본혜택가를 기준으로 판매가를 10~15% 무작위 인상**하고, 그 인상분만큼 기본쿠폰을 붙여 실제 결제가는 기존 수준으로 되돌리는 방식이다. 판매가는 기본혜택가에 1.10~1.15배 사이의 무작위 값을 곱해 정하고, 기본쿠폰은 (판매가 빼기 기본혜택가)로 두어 원 단위로 맞춘다. 정가는 원래 정가와 (판매가+1) 중 큰 값으로 둔다. 안전장치로 가격 변동률은 ±30% 이내, 판매가는 원가의 10배 이내로 제한하고 기본 마진은 25%를 가정한다.

기본혜택가(target_benefit) 우선순위 (차별화상회 2026-05-13):

우선	분기	1차 기본혜택가
1	지정가(AI)	ai_base_coupon_discount_amount 보유 → 그대로(업프라이싱 미적용)
2	HARDCODED	지정가(AI) 폴백(reserved/ai 누락 PID): 원가×(1+전체 중위마진) → 경쟁사 2차조정 → 업프라이싱
3	DAILY(메인)	당일 기본혜택가(어제 등록판매가-어제쿠폰) → 경쟁사 2차조정 → 업프라이싱
4	COMPETITOR	당일 혜택가 없는 신상품: 경쟁사가 → ×0.99 → 업프라이싱

-	BUNDLE_SECONDARY	묶음 비기준(같은 erp_code): 원가×(1+그룹 90일 마진 평균) → 업프라이싱
-	마스터가 유지	묶음 그룹 비기준 nonTOP(그룹에 TOP 존재): 차별화상회 마스터 가격 유지(등록 스킵)
-	NONE	혜택가·경쟁사가·원가 모두 없음 → 스킵

경쟁사 2차 조정(`adjust_for_competitor`, 차별화상회 2026-05-11 · 환산가 `unit_price` 기준): 5%+ 우위→매일 1% 인상(우위 5% cap) · 3~5% 우위→0.5% 인상(3% cap) · ≤3% 우위→유지 · 동등→-1%($\text{comp} \times 0.99$) · ≤2% 열위→동등화(comp) · >2% 열위→유지. cap·점진 인상은 min(점진가, cap)로 적용.

19.2 쿠폰 5종

쿠폰	유형/소스	대상	산출
기본쿠폰	PRODUCT / AI_BASE	전 상품	Up-price 차액(<code>base_coupon</code>), Phase2 Step6-1
재구매 쿠폰(A-4)	PRODUCT / AI_REC	6개월 2회+ 구매 상품	티어 차등(TOP/KEY 높게), Phase3
신규(미구매) 쿠폰(A-4b)	PRODUCT / AI_REC	미구매 상품(<code>purchase=0</code>)	min(혜택가×5%, 혜택가-원가), 1,000원 ↑
장바구니 쿠폰(B-4)	CART / AI_REC	직배송 고객	분위회귀+후처리(§ 19.3), 주1회·7일 유효
이탈/VIP 쿠폰	CART / AI_REC	직배송 이탈·우량	이탈 2건 고정 / VIP 추가

발급 API(Phase 3)

POST /coupons/ai-rec/create(스킵 생성, `coupon_type=PRODUCT|CART`) → `meta_coupon_number` → POST /coupons/ai-rec/issue(대상 발급). 장바구니쿠폰은 동일 (금액·최소주문) 조합별 스킵 1개 생성 후 해당 고객 발급.
`user_issue_limit=1`, 유효 7일, `issue_source_type=AI_REC`.

19.3 장바구니 쿠폰 후처리: `cart_coupon_orchestrator.py` (정본)

B-4 raw 출력에 다음 후처리를 적용한 값이 **최종 발급 금액**이다(적용 순서: 이탈 분리 → 체리 → 예산 → 반올림).

후처리	내용
이탈 오버라이드	이탈(A-2 tier 이탈%, DIRECT) 고객은 B-4 결과 대신 고정 2건: 8,000원/최소 15만 + 20,000원/최소 30만 . (이탈기준 90일)
휴면 환영	잠재(미구매) 고객도 동일 금액이나 발급 중단(차별화상회 2026-05-20) : 미구매(주문 0건)는 장바구니쿠폰 제외. 식별은 모니터링용 유지
체리피커	<code>downgrade_rate=1.0</code> : 하향 미적용 (라벨 <code>cherry_picker_downgrade</code> 만 유지, 차별화상회 2026-05-11)
예산(매출 2%)	전일 누적 사용률 <1.5% → ×1.1 상향 / >1.8% → ×0.9 하향 / >2.0% → 체리피커 발급 중단. 순 GP <13% → ×0.85(정책7 v4.2)
차별화상회 반올림	0→유지, 1~4→내림, 5→유지(500/5000), 6~9→올림. 쿠폰=1,000원 단위·clip[3,000~50,000], 최소주문=10,000원 단위·clip[150,000~500,000]

B-4 분위회귀(raw)는 고객의 과거 주문금액 분포에서 3사분위(Q3)와 2사분위(Q2)의 차이를 추가구매 유도분($\text{additional_purchase} = \max(Q3-Q2, 0)$)으로 잡는다. **운영 쿠폰 원안 = 추가구매 유도분 × 0.8**이며, 여기에 절대 금액 한도(3,000~50,000원)와 최소주문금액 대비 할인율 3.5~6.8% 범위로 clip한 뒤 반올림한다. 최소주문금액은 추가구매 유도분이 50,000원 미만이면 Q3+50,000원(유인 강화), 아니면 Q3로 두고 1,000원 단위로 반올림한 뒤 [15만~50만원]으로 clip한다. 주문이 3건 미만

으로 데이터가 부족하면 10,000원 / 200,000원 기본값을 쓴다. 이 원안에 위 후처리(이탈 분리, 체리, 예산, 반올림)를 적용한 값이 최종 발급액이 된다.

BEP는 검수 비용

손익분기 BEP = 추가구매 유도분 × 개인 GP율은 **실제 발급에 쓰이지 않는다**. 검수 비용용 컬럼 coupon_amount_gp100/120/140(= BEP × 1.0/1.2/1.4)에만 사용되어, 운영 쿠폰액을 GP 시나리오와 견주는 모니터링 지표로만 활용된다.

대상 필터

장바구니·이탈·VIP 쿠폰은 **직배송(DIRECT)** 고객만. 택배(PARCEL)는 build_b4_inputs 단계에서 사전 제외. 신규 30일 웰컴 필터·장기이탈 월2회 cap은 차별화상회 요청으로 비활성(2026-05-13/14).

20 운영: 실행 · 배포 · 모니터링 · 장애 Runbook

20.1 실행 (uv · 수동 실행 API)

```
# 스케줄러(운영 상시): systemd가 기동
ENV=prod uv run python -m workers.scheduler

# 전체 일배치 수동 실행 / 특정일 / 특정 Step
ENV=dev uv run python -m pipeline.daily_batch
ENV=dev uv run python -m pipeline.daily_batch --date 2026-06-18
ENV=dev uv run python -m pipeline.daily_batch --step 4 # A 모델만
```

배치 수동 실행 API (배치 EC2의 admin 서버, 포트 :8200. SSM 접속 없이 Phase를 수동 트리거. 같은 작업이 실행 중이면 409 반환):

메서드 · 경로	동작
GET /admin/status	현재 실행 중인 Phase 목록 조회
POST /admin/phase1	Phase 1 (오전 집중 산출: 적재·드리프트·경쟁사가·가격계획·팩트·C-3)
POST /admin/phase2	Phase 2 (가격 등록 + 기본쿠폰)
POST /admin/phase3	Phase 3 (B/A 모델·개인화/장바구니 쿠폰 + S3 업로드)
POST /admin/phase2and3	Phase 2 → Phase 3 순차 실행
POST /admin/report	검수 리포트 수동 발송

Phase 엔드포인트는 batch_date(YYYY-MM-DD) 쿼리 파라미터를 받으며, 생략 시 당일(KST)로 실행된다.

20.2 서버 접속 (AWS SSM · 스마트 푸드 네트워크 VPC)

```
aws ssm start-session --target i-0ebecdd5984cc878a --profile sfm # PROD 배치
sudo journalctl -u atom-scheduler -f # 스케줄러 로그
sudo systemctl status atom-scheduler atom-s3-consumer # 상태
```

서비스	호스트	역할
-----	-----	----

atom-scheduler	배치 EC2	3-Phase 스케줄러 + admin :8200
atom-s3-consumer	배치 EC2	S3 이벤트(SQS) 소비·적재
atom-api (:8100)	API EC2	추천/쿠폰/프로파일링 API
atom-sqs	API EC2	신규/수정 고객 FIFO 소비 → 즉시 추천(실시간)
profiling_worker	API EC2	24시간 식자재 추출(C-1) 상시 처리 · 신규 가입자 재스캔
API_scheduler	API EC2	07:00 하트비트 Slack · 기동/크래시 알림 · 일배치 트리거

20.3 배포 (CodePipeline)

```
# 배치 로컬 운영/개발 반영
git push origin main          # PROD 파이프라인
git push origin main:dev      # DEV 파이프라인
# B 모델(editable 설치)은 batch EC2에서 git pull 만으로 반영
```

API/Batch는 AWS CodePipeline(*-dev/*-prod) → CodeDeploy 자동 배포. appspec.yml/buildspec.yml 포함.

20.4 운영 게이트 플러그 (config/settings.py)

플러그	역할
INTEGRATION_ENABLED	차별화상회 외부 호출(가격등록·쿠폰발급 API) 마스터 스위치. false면 산출만, 외부 반영 차단
INSPECTION_WORKFLOW_ENABLED	검수 메일/승인 게이트. false면 폴링 없이 자동 통과
CART_COUPON_ISSUANCE_ENABLED	장바구니쿠폰 발급 on/off
LMS_SEND_ENABLED	LMS 실제 발송 on/off

20.5 모니터링

- **Slack 알림:** 07:00 하트비트, Phase 시작/완료/에러, 지연(@channel, Phase1>2h·Phase3>3h), 승인 대기/완료/반려/타임아웃, 클린업.
- **진행 추적 DB:** batch_runs(실행 단위), batch_progress_logs(단계 진행률·Slack 동기), event_logs(웹훅/SQS 수신), customer_pipeline_status(고객별 상태).
- **검수:** 14:00 검수 리포트(상위 500 가격), cart_coupon_results GP100/120/140·raw 컬럼으로 사후 모니터링.

20.6 장애 Runbook

증상	점검 → 조치
Phase 1 enrich 0% 정체	직전 세션 트랜잭션 경합 → enrich는 fresh 세션 분리 적용됨. 장기 시 스케줄러 재기동, --step 2 수동 재실행
가격 미반영(다음날)	승인 여부 확인(미승인 시 Phase2/3 스킵) → 승인 후 --step 5,6 수동, 또는 익일 정상화
쿠폰 미발급	INTEGRATION_ENABLED / CART_COUPON_ISSUANCE_ENABLED 확인 → 차별화상회 발급 API 응답 로그 확인
LMS 미발송	LMS_SEND_ENABLED + lms_message_logs 이력 확인(중복방지로 스킵됐는지)

재시작 후 Phase 3 스킵	phase2_done 복원 실패 가능 → price_registration_history SUCCESS 확인, 필요 시 수동 Phase 3
경쟁사가 누락	크롤러 webhook 수신 · competitor_prices 확인 → 미수신 시 DAILY/원가 폴백으로 자동 진행

20.7 데이터 보존

- customer_product_facts: **당일치만**(매 Phase 3 과거일 삭제). 복구는 재산출 또는 RDS PITR.
- RDS 자동 스냅샷 7일(PITR). 산출 이력 테이블(recommend_results, coupon_records, cart_coupon_results, *_history)은 일자 누적 보존.

APPENDIX

부록

인도 범위, 연동 인터페이스(프로파일링·크롤링·승인), 자격증명·시크릿 관리, 통합 용어 집.

A 인도 범위: 무엇이 전달되고 무엇이 아닌가

구성요소	인도 형태	비고
추천·가격·쿠폰 일배치	코드 전체	3-Phase 일배치 · S3 이벤트 수신 · A 모델 · 쿠폰 발급. 운영 메인
추천·쿠폰 API 서버	코드 전체	추천/쿠폰 API · 신규고객 처리 · 프로파일링 결과 수신 API
AI 모델 라이브러리	코드 전체	연관구매·구매주기·장바구니쿠폰·서브클러스터 모델. 일배치에 내장(editable)
매장 메뉴 크롤러	코드 전체	네이버·카카오·다이닝코드·식신 메뉴 수집 (\$ 12.6)
경쟁사 가격 크롤러	코드 전체	식봄·배민상회 가격 수집·환산 (\$ 12.7)
메뉴 프로파일링 서비스	연동 인터페이스	Weraser 운영. 메뉴→식자재(C 모델, Gemini). 연동 규격은 부록 B
관리 대시보드 (승인 게이트)	연동 인터페이스	승인 상태 API(/api/v1/approvals/...) 계약만

의존성 메모

AI 모델 라이브러리는 일배치에 **내장(editable)** 설치되어 함께 전달되므로, 인도 시 두 구성요소를 같은 상위 경로에 두고 의존 경로를 확인할 것. 두 크롤러(매장 메뉴·경쟁사 가격)는 하나의 저장소에 두 앱으로 구성되며 코드 전체가 인도된다. 메뉴 프로파일링 서비스·관리 대시보드는 Weraser가 운영하며, 코드 없이 **연동 규격(부록 B)만으로 동작한다**.

B 연동 인터페이스 (서비스 간 계약)

B.1 프로파일링 서비스 → ATOM API

프로파일링 서비스는 DB에 직접 접근하지 않고 POST /api/v1/profiling/save로만 결과를 저장한다(VPC 피어링, 다건 batch).

```
POST /api/v1/profiling/save
{
  "profiles": [{
    "customer_id": "12345", "store_name": "○○식당", "address": "", "source": "",
    "business_type": "한식", "analysis_status": "STORE_CONFIRMED",
    "menu_names": ["김치찌개", ...], "all_ingredients": ["돼지고기", ...],
    "matched_count": 12, "extracted_count": 3,
    "matched_results":
    [{ "menu_name", "matched_recipe", "ingredients", "method", "confidence", "similarity" }],
    "extracted_results": [{ "menu_name", "ingredients": [...], "matched_recipe", "method", "confidence" } ]
  }]
}
→ 200 { "saved": N, "total": M, "errors": [] }
```

B.2 크롤러 → ATOM API (완료 콜백)

```
POST /api/webhook/crawl-complete
{ "type": "menu" | "price", "customer_id": "...", "status": "success|failed|partial",
  "run_id": "...", "results": [...], "summary": {...} }
→ 202 { "status": "accepted", ... } (후속 파이프라인은 백그라운드 실행)
```

POST /api/webhook/customer-event # SQS 고객 이벤트 HTTP 보조 수신

B.3 크롤러 → 경쟁사 가격 (DB)

경쟁사 가격 크롤링 결과는 crawler_{env} DB에 적재되고, 배치 Step0가 읽어 competitor_prices로 정규화한다.

항목	내용
소스	price_item_urls (크롤링 대상, is_no_match=false) · price_crawled (수집가)
비교 기준	환산가 unit_price 우선, 없으면 price 폴백(차별화상회 2026-05-04 합의)
정규화 산출	competitor_prices (item_code, batch_date, lowest_price, lowest_unit_price, lowest_source)

C 자격증명 · 시크릿 관리 (마스킹 안내)

원칙

본 문서의 모든 자격증명·비밀 값은 마스킹되어 있다. 실값은 AWS Secrets Manager에서 로드되며, 인도 시 별도 보안 채널(예: 1Password/대면 전달/봉인)로만 공유한다. 본 문서에는 절대 포함하지 않는다.

분류	본 문서 표기	실제 출처
DB 접속(호스트/포트/DB명)	포함(운영 식별자)	-
DB 사용자/비밀번호	(마스킹)	Secrets Manager atom/{env}/db
차별화상회 발급 API Basic Auth	(마스킹)	Secrets Manager atom/{env}/config
Slack Webhook URL	(마스킹)	Secrets Manager atom/{env}/config

Gemini API Key(프로파일링)	(마스킹)	Secrets Manager atom/{env}/config
EC2 instance ID / Private IP / S3 버킷 / RDS 호스트	포함	스마트 푸드 네트워크스 소유 운영 식별자(개발팀 운영 필요)

애플리케이션은 기동 시 atom/{env}/config(API·Webhook·Key 등)와 atom/{env}/db(DB 자격증명)를 로드한다. 로컬 개발은 .env(.env.example 참고, 비밀 값 제외 커밋)로 주입한다.

D 통합 용어집

용어	정의
Up-price(동적 프라이싱)	판매가를 혜택가×(1.10~1.15)로 인상하고 차액을 기본쿠폰으로 환원
기본혜택가(target_benefit)	업프라이싱 기준가 = 최종 결제 목표가. 우선순위 지정가(AI)→HARDCODED→DAILY→COMPETITOR
혜택가(new_benefit_price)	판매가 - 기본쿠폰 = 고객 최종 결제가
recommend_type	REPURCHASE(재구매) / RECOMMENDATION(신규). A-6 _classify
price_tier(4-Tier)	고객별 매출순 4등분: TOP/KEY/STANDARD/TAIL (A-5)
w1_score	B-2 연관구매 점수(0~1)
d_day / urgency_score	B-3 구매 예상까지 남은 일수(음수=지연) / 긴급도(>1=지연)
P_avg / O_avg	개인 구매주기 중앙값 / 동적 윈도우 기준 평균간격(Clamp 3~7)
BEP(손익분기)	추가구매 유도분 × 개인 GP율. 운영 발급엔 미사용, 검수 비교 컬럼(gp100/120/140) 전용
GP(Gross Profit)율	(판매가-원가-할인)/판매가. 예산 가드(순GP<13%→×0.85)
segment	A-1: FC자사 / FC타사 / 일반식당
customer_tier	A-2: 이탈-위험/확실/확실(장기), 잠재(미구매) 등
체리피커	A-3: 쿠폰 사용률이 높은 고객(현재 하향 미적용·라벨만)
FC(프랜차이즈)	자사 FC=fc_customers 매핑, 타사 FC=fc_headquarters 영업표지
SKU 매핑(C-3)	식자재→상품. 4-Layer: sfn_brand→sfn_name→personal→industry
shipping_type	DIRECT(직배송)/PARCEL(택배). 장바구니·이탈·VIP·LMS는 DIRECT만
planned_prices	가격계획 산출 테이블(Phase1) → Step5 등록(Phase2)
승인 게이트	Phase2 전 검수 승인(5분 폴링, 17:30 마감)
Phase 1/2/3	08:00 산출 / 16:10 가격등록·기본쿠폰 / 18:10 모델·개인화·장바구니·S3

- 이상, ATOM 통합 산출물 (초안) -
본 문서의 자격증명 값은 마스킹되어 있습니다. · 대외비